Uninformed Search

CSCI3230 (ESTR3108) | Tutorial 9

Wenxin Fan

Email: wxfan25@cse.cuhk.edu.hk

Office: Pentecostal Mission Hall Complex High Block, IMIXR Lab

Department of Computer Science and Engineering
The Chinese University of Hong Kong

November 2026



Learning Goals

1. Explain what a **Fringe** contains and how **Expansion** works.
2. Use a **Queue**, **Stack**, or **Priority Queue**.
3. Trace the steps of **BFS, UCS, DFS, DLS, and IDDFS**.
4. Explain how the algorithms select nodes, perform Goal Tests, and update the Fringe.

Today's Focus: **Definitions** $\rightarrow$ **Data Structures** $\rightarrow$ **Worked Examples** $\rightarrow$ **Pseudocode**

Search Terminology

State Space Graph
A mathematical representation of states and legal actions. An individual state occurs once in the graph.

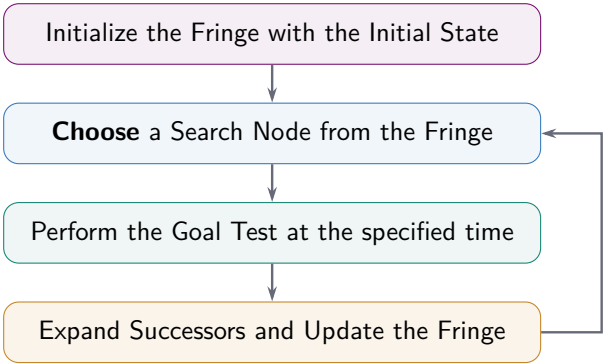
Search Tree
A tree of candidate *paths* from the initial state; the same state may appear along multiple paths.

Fringe
Search nodes that are waiting to be expanded. Also called the *Frontier* in many implementations.

Expansion and Search Strategy
Expansion picks a node from the Fringe and generates its successors. A search strategy decides *which* node comes next.

Key Concepts: State Space Graph → Search Tree → Fringe → Expansion.

One General Search Procedure

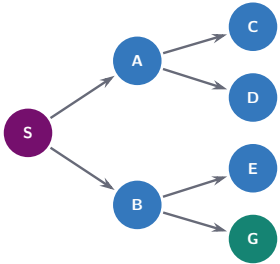


Different Strategies Use Different Rules for Choosing a Search Node.

Breadth-First Search (BFS): Definition

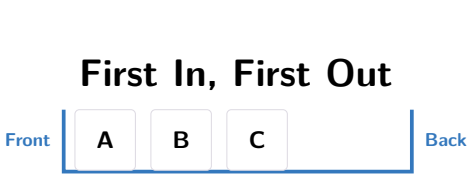
Definition
BFS expands the Root Node first, then its successors, then their successors: it explores the Search Tree **level by level**.

- Process the **shallowest** available Search Nodes first.
- Implement the Fringe using a **FIFO Queue**.
- Use **Goal Test upon Generation** when a new node is generated (after checking the Initial State).



BFS Goes Level by Level: Depth 0, Then 1, Then 2, ...

Queue: What Does FIFO Mean?

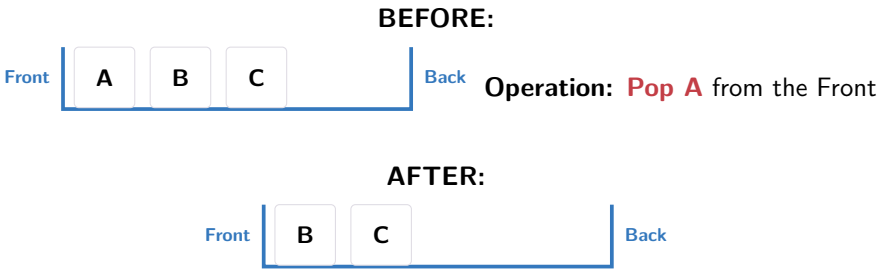


POP / Dequeue Remove from the Front (the left side). Here, A leaves first.

PUSH / Enqueue Add to the Back (the right side). New items wait their turn.

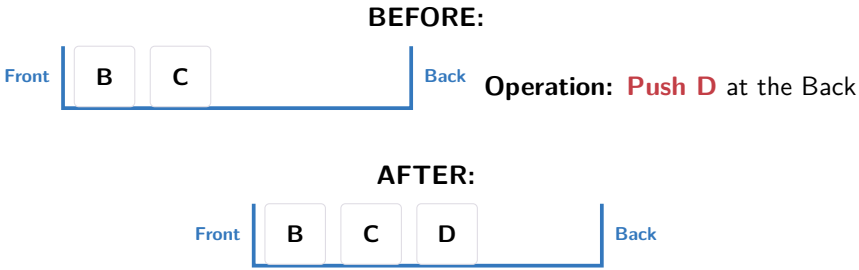
In Every Queue Diagram: Left = Front (Pop); Right = Back (Push).

Queue Operation 1: Pop from the Front



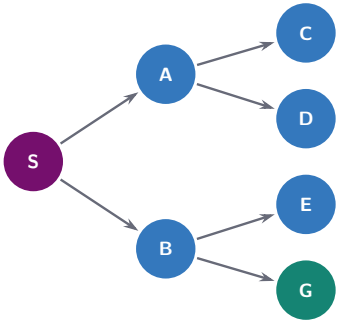
A Is Removed. The Queue Now Contains Only B and C.

Queue Operation 2: Push at the Back



B Will Be Popped Next; Newly Added D Waits Behind C.

BFS Worked Example: Search Tree



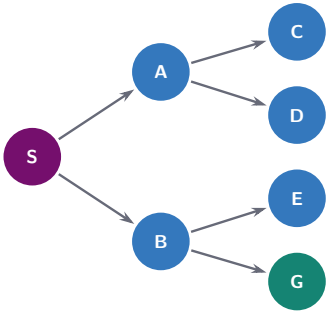
Search Problem
Initial State: **S**
Goal Test: **Is the State G?**

Successor Order
Alphabetical Order
S: A, B; A: C, D; B: E, G

All arcs are directed. The Root Node is at depth 0.

Track the Fringe / Queue After Every Expansion.

BFS Trace: Initialize and Expand S



Start Fringe = [S]; Discovered = {S}

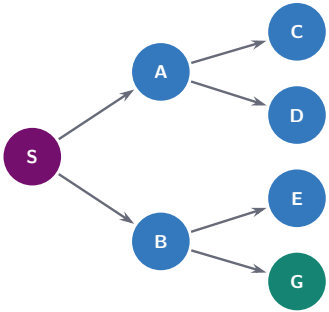
Pop S Generate A, then B; both are new states.

Fringe after Expansion:

Front | A | B | Back

A is at the Front and will be selected next.

BFS Trace: Expand A



Before Pop Fringe = [A, B]

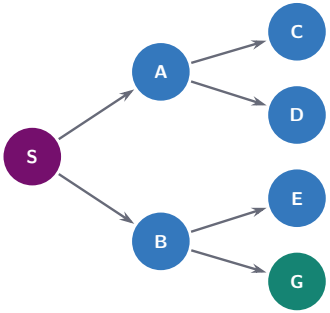
Pop A Generate C, then D. Add both at the Back.

Fringe after Expansion:

Front | B | C | D | Back

B was discovered earlier than C and D.

BFS Trace: Generate the Goal



Before Pop Fringe = [B, C, D]

Pop B Generate E; then generate **G**.

Goal Test G is the Goal. **Stop immediately upon Generation**.

BFS does not need to pop C, D, E, or G first in this version.

Returned Path: S → B → G.

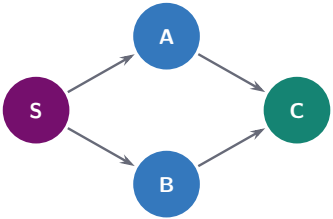
Breadth-First Search (BFS): Pseudocode

```
function BFS(start):
  if GoalTest(start): return [start]
  Fringe ← FIFO_Queue([start])
  Discovered ← {start}
  Parent[start] ← None
  while Fringe is not empty:
    u ← PopFront(Fringe)
    for v in Successors(u):
      if v not in Discovered:
        Discovered.add(v)
        Parent[v] ← u
        if GoalTest(v): return Path(v)
        PushBack(Fringe, v)
  return Failure
```

Fringe Operations
PopFront: FIFO order
Discovered.add(v): no duplicate enqueue
GoalTest(v): check generated successors

The initial state is checked separately before the loop.

BFS: Why Record Discovered States?



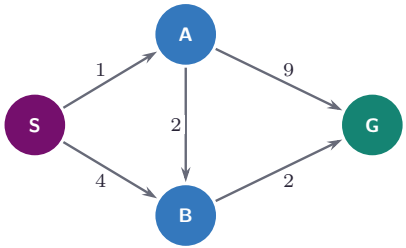
- Expand S: Discover A, B.
- Expand A: Discover C; mark C.
- Expand B: See C again, but **do not enqueue it again**.

Mark a New State When It Is Enqueued, Not Only When It Is Expanded.

Uniform-Cost Search (UCS): Definition

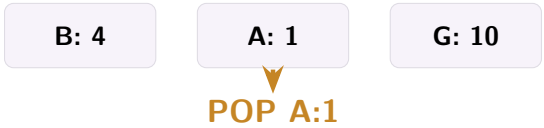
Definition
UCS expands the Search Node with the **lowest Path Cost**, not necessarily the shallowest node.

- Keep $g(n)$, the cost from the Initial State to node n.
- Use a **Priority Queue** for the Fringe.
- A cheaper discovered path can **replace an earlier path**.



UCS Selects the Candidate with the Lowest Cumulative Path Cost $g(n)$.

Priority Queue: Smallest Cost First

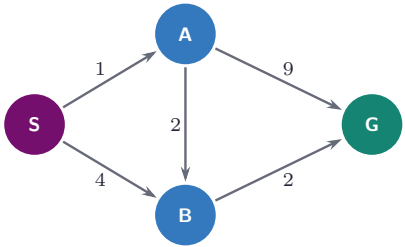


FIFO Queue
Select the **oldest** enqueued node.

Priority Queue
Select the node with the **lowest** $g(n)$, regardless of arrival time.

A Priority Queue Is Not a FIFO Queue: Its Priority Depends on Path Cost.

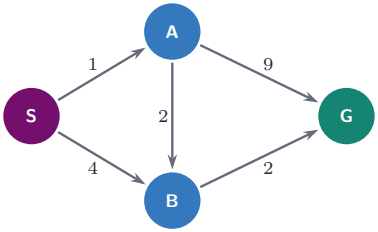
UCS Worked Example: Try It First



Search Problem
Initial State: S
Goal State: G
Edge Labels: Step Cost

1. Which Node Is Popped after S?
2. Can the Path Cost to B improve?
3. When Do We Stop at G?

UCS Trace: Pop S

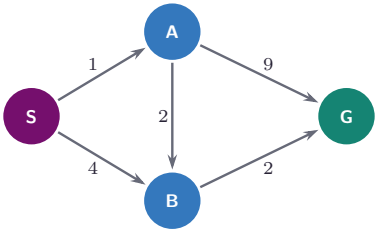


Pop S:0 Generate A with cost 1; B with cost 4.

Fringe Node	g	Parent
A	**1**	S
B	4	S

Next Pop: A:1

UCS Trace: Pop A and Relax B



Pop A:1 A→B costs 2; A→G costs 9.

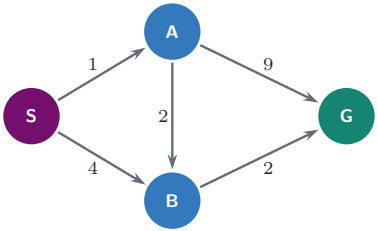
$$g(B) : 4 \rightarrow 1 + 2 = 3$$

Fringe Node	g	Parent
B	**3**	**A**
G	10	A

Next Pop: B:3

Relaxation: Update Both the Best Path Cost and the Parent.

UCS Trace: Pop B and Improve G



Pop B:3 B→G costs 2.

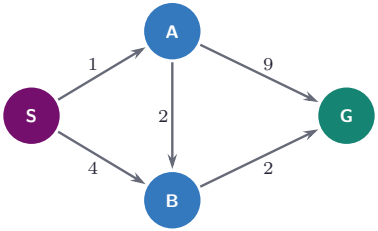
$$g(G) : 10 \rightarrow 3 + 2 = 5$$

Fringe Node	g	Parent
G	**5**	**B**

Next Pop: G:5

Discovering G at Cost 10 Was Not Enough to Stop the Search.

UCS Trace: Pop the Goal and Return the Path



Pop G:5 Goal Test is successful **when G is popped**.

Parent Links
G←B←A←S

Solution
S → A → B → G
Total Path Cost = **5**

UCS Returns G When It Has the Lowest Cost in the Fringe.

UCS Pseudocode: Cost Update and Goal Test

```
function UCS(start):
  Best[start] ← 0; Parent[start] ← None
  Fringe ← PriorityQueue((0,start))
  while Fringe is not empty:
    (g,u) ← PopMin(Fringe)
    if g is outdated: continue
    if GoalTest(u): return Path(u)
    for (v,step) in Successors(u):
      newCost ← g + step
      if newCost < Best[v]:
        Best[v] ← newCost
        Parent[v] ← u
        Push(Fringe,(newCost,v))
  return Failure
```

Goal Test and Cost Update

PopMin: smallest g

GoalTest(u): goal checked on pop

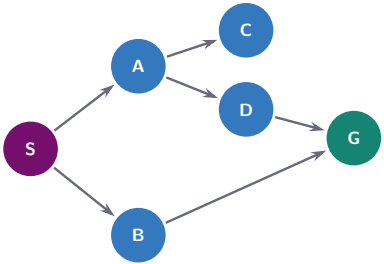
newCost $<$ **Best[v]**: relax edge

Parent[v]: updated path

Depth-First Search (DFS): Definition

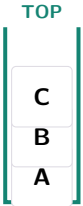
Definition
DFS expands the **deepest Search Node in the current Fringe**. It follows one branch before exploring another.

- Continue deeper whenever possible.
- **Backtrack** when a branch cannot continue.
- Implement with **Recursion** or a **LIFO Stack**.



DFS Goes Deep First; It Does Not Explore the Search Tree Level by Level.

Stack: What Does LIFO Mean?



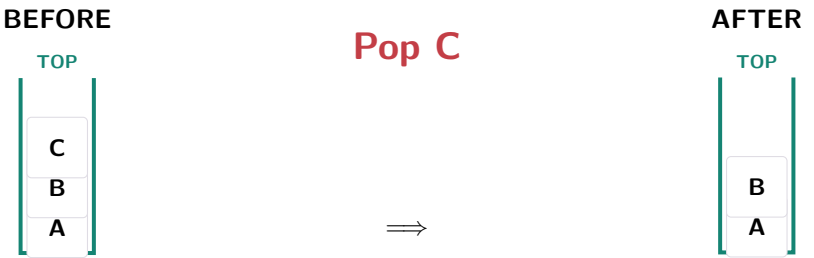
Last In, First Out
A Stack has only one active end: the **Top**.

PUSH Add an item to the Top.

POP Remove the item currently at the Top.

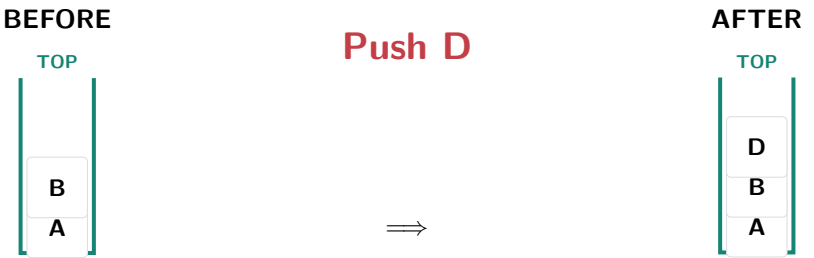
Here, C Was Pushed Last, So C Is Popped First.

Stack Operation 1: Pop from the Top



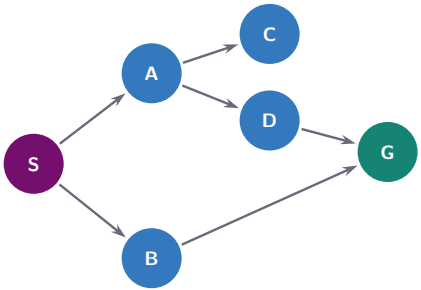
C Is Removed: the Next Item at the Top Is B.

Stack Operation 2: Push onto the Top



D Becomes the Top: It Will Be Popped Before B and A.

DFS Worked Example: One Branch at a Time

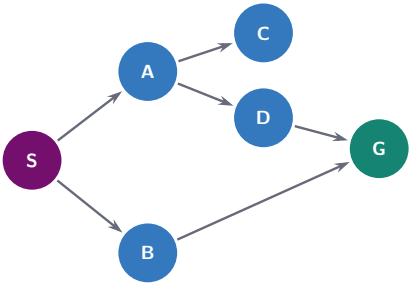


Search Problem
Start S; Goal G
Neighbors are visited alphabetically.

1. Enter S $\rightarrow$ A $\rightarrow$ C.
2. C has no successors: **Backtrack to A**.
3. Continue A $\rightarrow$ D $\rightarrow$ G.

Discovery Order: S, A, C, D, G **Returned Path: S $\rightarrow$ A $\rightarrow$ D $\rightarrow$ G.**

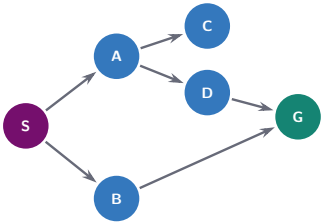
DFS Backtracking: Why Is C Not on the Solution Path?



1. Explore C first because C comes before D.
2. **No new Successors** from C; return to A.
3. Resume the unfinished loop inside A.
4. Explore D, then find G; **stop the search**.

Visited Order $\neq$ Returned Path: Backtracking Can Discard Unsuccessful Branches.

Iterative DFS: Make the Stack Visible



Operation	Stack (Bottom → Top)
Initial	[S]
Pop S; Push B, A	[B, A]
Pop A; Push D, C	[B, D, C]
Pop C	[B, D]
Pop D; Push G	[B, G]
Pop G	Goal Found

To Visit A Before B, Push B Before A: the Last Pushed Item Is Popped First.

Recursive DFS: Pseudocode and Backtracking

```
function DFS(u):
  Visited.add(u)
  if GoalTest(u): return [u]
  for v in Successors(u):
    if v not in Visited:
      result ← DFS(v)
      if result is Success:
        return [u] + result
  return Failure
```

Recursion and Backtracking
Visited.add: avoid loops
DFS(v): go deeper
return Failure: backtrack and try the next neighbor

This is goal-directed DFS, rather than merely listing all visited states.

Iterative DFS: Pseudocode and Push Order

```
function DFS_Iterative(start):
  Fringe ← Stack([start])
  Discovered ← {start}
  Parent[start] ← None
  while Fringe is not empty:
    u ← PopTop(Fringe)
    if GoalTest(u): return Path(u)
    for v in Reverse(Successors(u)):
      if v not in Discovered:
        Discovered.add(v)
        Parent[v] ← u
        PushTop(Fringe,v)
```

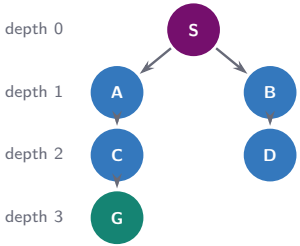
Stack and Discovery
PopTop: LIFO order
Reverse: push later neighbors first
Discovered: avoid duplicate pushes and preserve Parent

A Stack-based graph-search implementation can use a different visit order from a fully recursive DFS when paths converge.

Depth-Limited Search (DLS): Definition

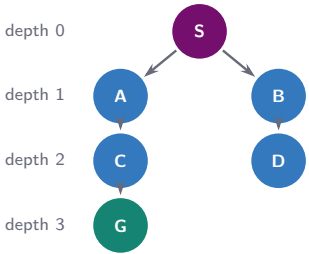
Definition
DLS is Depth-First Search with a **pre-determined Depth Limit** ℓ.

- The Root Node starts at depth 0.
- A node at depth ℓ can be goal-tested but is **not expanded further**.
- If a Goal lies deeper than ℓ, this run will not reach it.



DLS = DFS + Depth Limit.

DLS Worked Example: Depth Limit 2



Set $\ell = 2$ Start at S (depth 0).

Go Deep S $\rightarrow$ A $\rightarrow$ C. C is at depth 2.

Depth Cutoff Do **not** expand C $\rightarrow$ G, because G is at depth 3.

Backtrack Continue with other available branches.

A Depth Limit Is a Cutoff, Not a Goal: Depth 2 Nodes May Still Be Goal-Tested.

DLS Pseudocode: Goal Test Before Depth Cutoff

```
function DLS(u, depth, limit, path):
  if GoalTest(u): return [u]
  if depth == limit: return Cutoff
  for v in Successors(u):
    if v not in path:
      ans ← DLS(v, depth+1, limit,
                path ∪ {v})
      if ans is Success: return [u]+ans
  return Failure
```

Goal Test and Depth Cutoff

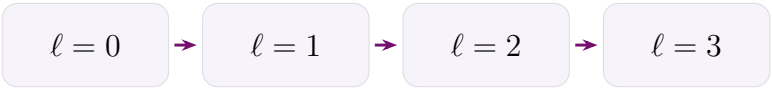
GoalTest first: goal may be exactly at the limit

Cutoff next: avoid deeper expansions

path: avoid cycles on the current branch

Iterative Deepening Depth-First Search (IDDFS): Definition

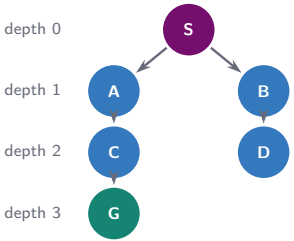
Definition
IDDFS repeatedly runs **Depth-Limited Search**, increasing the Depth Limit until a goal is found.



- **Restart from the Initial State** after increasing the limit.
- Previously cut-off branches become accessible in later iterations.

IDDFS Does Not Resume the Previous DLS Run; It Starts a Fresh Run.

IDDFS Worked Example: Limit 0, 1, 2, 3



Limit	Visited in This Run
$\ell = 0$	S; no Goal
$\ell = 1$	S, A, B; no Goal
$\ell = 2$	S, A, C, B, D; no Goal
$\ell = 3$	S, A, C, **G**; stop

Every Row Starts Again from S: Previously Visited States Must Not Block a Later Run.

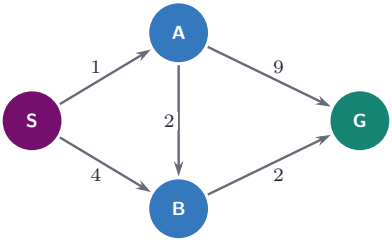
IDDFS Pseudocode: Restart with a New Limit

```
function IDDFS(start, maxDepth):
  for limit = 0 to maxDepth:
    path ← {start}
    ans ← DLS(start,0,limit,path)
    if ans is Success: return ans
  return Failure
```

Fresh Search at Each Limit
Fresh path: no stale visited set from an earlier limit
New DLS call: start over at depth 0

DLS Is the Building Block; IDDFS Repeats It with a Larger Limit.

Practice: Compare BFS, UCS, and DFS



Rules
Start S; Goal G; directed arcs; alphabetical successor order.

1. What path does **BFS** return?
2. What path does **UCS** return?
3. What path does **Recursive DFS** return?
4. Why are the paths not always the same?

Practice Answers: Different Selection Rules

Strategy	Returned Path	Path Cost
BFS	$S \to A \to G$	$1 + 9 = 10$
UCS	$S \to A \to B \to G$	$1 + 2 + 2 = 5$
Recursive DFS	$S \to A \to B \to G$	$1 + 2 + 2 = 5$

- BFS selects by **Depth**, UCS by **Path Cost**.
- DFS follows the **specified Successor Order**; the cheap path here is a coincidence.

Selection Rule $\to$ Expansion Order $\to$ Returned Path.

Three Questions to Remember

1. Which Fringe?
FIFO Queue, LIFO Stack, or Priority Queue?

2. Which Node Next?
Shallowest, lowest Path Cost, or deepest?

3. When Do We Stop?
Goal Test on Generation or on Pop? Depth cutoff?

Questions?